

PORTFOLIO

개발자 김준규

PORTFOLIO

김준규

ANDROID DEVELOPER

사용자 흐름을 안정적인 모바일 앱 구조로 설계하는 Android 개발자 김준규입니다.

Android 앱에서 단순히 화면을 구현하는 것보다, 데이터가 들어오고 상태가 바뀌며 사용자가 피드백을 받는 전체 흐름을 중요하게 생각합니다. Jetpack Compose 기반 프로젝트에서 화면, ViewModel, Repository, API 연동 구조를 분리해 구현했고, FCM 알림과 WebSocket 실시간 통신처럼 사용자 경험에 직접 영향을 주는 기능을 다뤘습니다.

AWARDS



2026 삼성청년SW·AI아카데미 특화 프로젝트 1위 수상

CONTACT



010-2498-5282



rlawnsrb298@kakao.com



github.com/uleh298-crypto

EDUCATION & EXPERIENCE

2025 ~ 현재	삼성청년SW·AI아카데미 14기 수료
2023 ~ 2025	연세대학교 소프트웨어공학과 졸업
2021 ~ 2022	학점은행제 이수 후 편입
2019 ~ 2020	대한민국 육군 병장 만기 제대
2018 ~ 2019	호서대학교 재학

KEY SKILLS

Kotlin

DTO, Mapper, Repository, ViewModel 구조 분리 및 로직 구현



Jetpack Compose

상태 기반 UI 구성, ViewModel과 UI State를 연결한 화면 상태 관리



What's Your ETF

기간: 2026.02.16 ~ 2026.04.03 / 인원: 5명

PROJECT OVERVIEW

투자 성향 기반 ETF 탐색 및 관리 Android 서비스

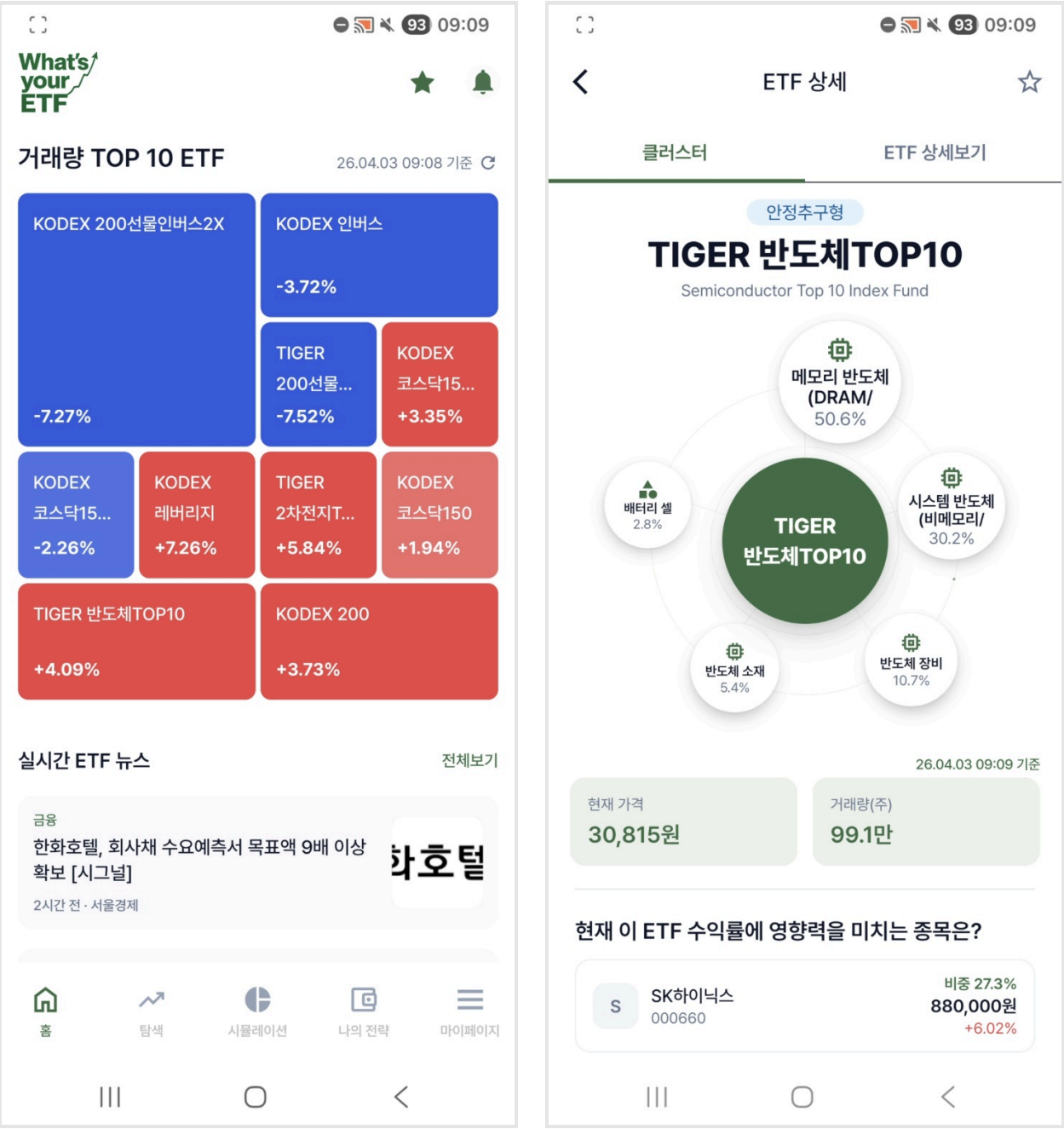
- **목표:** 사용자의 투자 성향과 ETF 데이터를 기반으로 ETF 탐색, 관심 ETF 관리, 개인화된 금융 정보 제공
- **배경:** ETF 상품은 정보량이 많고 초보 사용자가 비교하기 어렵기 때문에, 모바일 환경에서 쉽게 탐색하고 관리할 수 있는 흐름이 필요했음

MY ROLE & CONTRIBUTION

- **주요 담당:** ETF 상세, 관심 ETF, 마이페이지, 나의 전략, 뉴스 피드, 푸시 알림, 온보딩 영역 구현
- **아키텍처 설계:** Retrofit 기반 API 연동과 Repository / Mapper / ViewModel 구조 연결
- **데이터 관리:** FCM 토큰 저장 및 로그아웃 시 서버 전송 흐름 구현
- **성능 개선:** 관심 ETF 상태 동기화와 불필요한 API 호출 구조 개선

VISUAL PRESENTATION

실제 모바일 화면 배치 영역



What's Your ETF

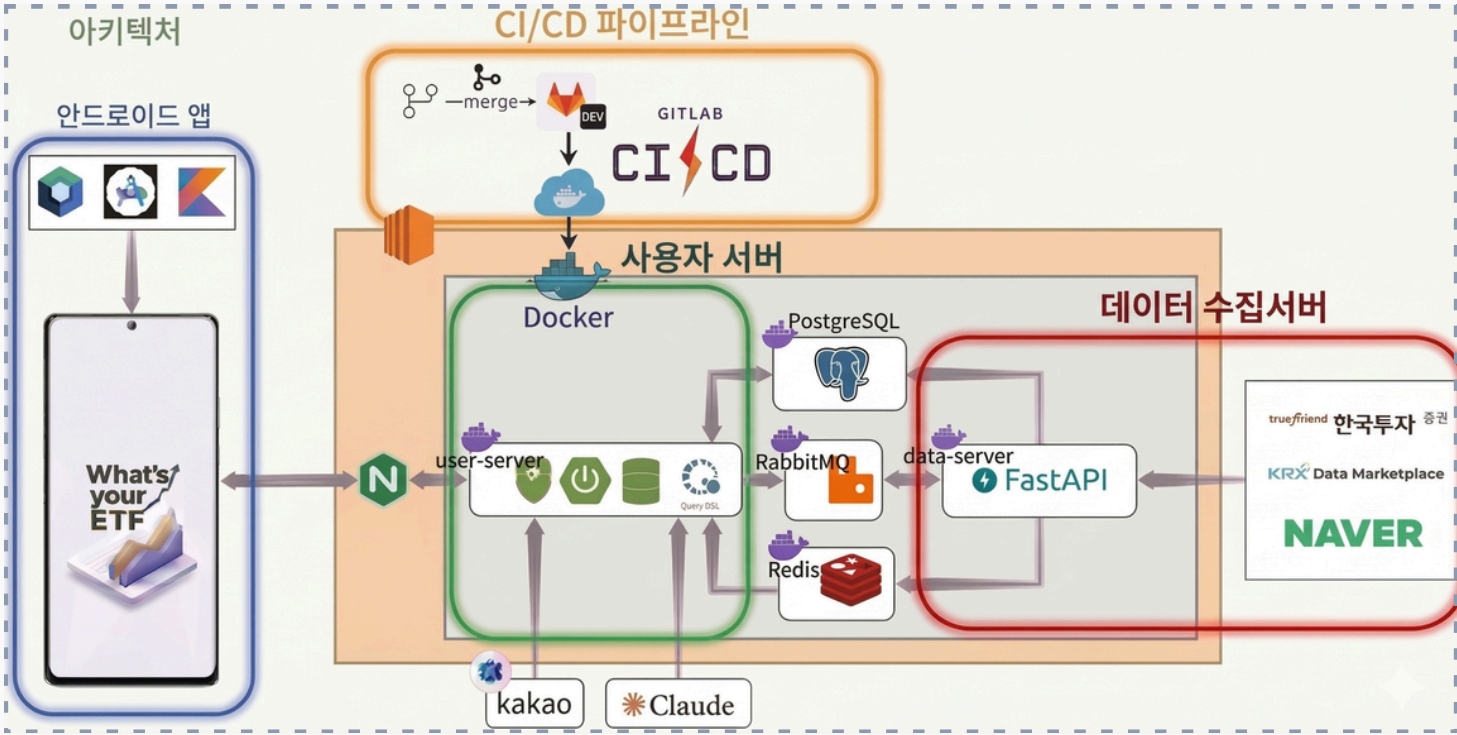
TECHNOLOGY

CONTEXT OF APPLICATION

Kotlin / Compose	금융 데이터를 리스트, 상세, 상태별 UI로 표현하기 위해 선언형 UI 프레임워크 적용
MVVM / Clean Arch	API 응답, 도메인 모델, UI State를 계층별로 분리하여 유연한 기능 변경 보장
Retrofit / Mapper	ETF 목록, 상세, 관심 ETF, 마이데이터 API를 도메인 계층에 맞게 매핑 및 연동
DataStore / FCM	FCM 토큰을 안전하게 로컬 저장하고, 로그인/로그아웃 라이프사이클에 맞게 서버 동기화
BaseResponse 처리	반복되는 응답 형식을 캡슐화하기 위해 공통 함수 및 BaseRepository 설계

KEY IMPLEMENTATIONS

- **상태 동기화:** 관심 ETF 등록/삭제 API 연동 및 목록 화면에서 UI State 기반 관심 여부 실시간 표시
- **마이데이터:** 보유 ETF 전체보기 및 마이데이터 API 연동
- **실시간성:** Top 10 ETF 데이터 폴링 및 당겨서 새로고침 기능 추가
- **예외 처리:** 공통 세션 만료 처리와 로그아웃 시 FCM 토큰 해제 안내 메시지 개선



What's Your ETF

PROBLEM STATEMENT

[이슈] 관심 ETF 여부 확인 API 반복 호출로 인한 성능 저하

ETF 목록 화면 진입 시, 각 ETF마다 관심 여부를 확인하는 API가 반복 호출되어 네트워크 요청이 과도하게 발생하는 구조적 문제가 발생했습니다. (목록 크기만큼 호출 수 비례 증가)

RESOLUTION PROCESS

1. 문제 진단

목록 화면에서 필요한 데이터 구조를 분석하고, 서버 응답의 ETF 목록 데이터에 이미 `isFavorite` 필드가 포함되어 있음을 확인
2. 전략 수립

개별 ETF마다 `/favorites/etfs/{ticker}/check` API를 반복 호출하지 않고, 목록 조회 응답의 필드를 직접 활용하도록 구조 변경
3. 기술 적용

Repository와 Mapper 계층에서 `isFavorite` 값을 UI State로 매핑하여 전달. Compose 화면은 추가 API 호출 없이 목록 상태를 기반으로 관심 여부 렌더링
4. 검증

관심 ETF 등록/삭제 후 목록 및 상세 화면에서 상태가 정상적으로 동기화되고, 네트워크 요청이 획기적으로 감소했음을 검증

PERFORMANCE COMPARISON

구분	BEFORE	AFTER
호출 방식	ETF 개수만큼 개별 API 반복 호출	목록 API 1회 호출로 통합 처리
네트워크 요청	최대 20+회 발생	단 1회 발생
화면 렌더링	개별 API 응답 대기로 인한 지연	목록 로딩과 동시에 즉시 렌더링

Before	After
목록 API (1회) + 개별 Check API (N회) 총 N+1회 호출	목록 API (isFavorite 포함) 단 1회 호출로 해결

What's Your ETF

PROJECT ACHIEVEMENTS

- **SSAFY 특화 프로젝트 1위 수상:** 우수한 기획력과 안정적인 기술 구현력을 인정받아 최우수 프로젝트로 선정
- **금융 도메인 역량 확보:** 대용량 금융 데이터를 모바일 환경에 최적화하여 연동하는 구조적 설계 경험 확보
- **핵심 사용자 기능 구현:** 관심 ETF, 마이페이지, 뉴스, 푸시 알림 등 사용자 리텐션과 개인화에 직결된 핵심 기능 성공적 구축
- **구조적 개발 경험:** API 응답, 로컬 저장소(DataStore), UI State, 사용자 피드백을 유기적으로 연결하는 설계 역량 획득

성과 요약: 금융 도메인 데이터 최적화 및 구조적 설계 역량 입증

RETROSPECTIVE

구분	내용
아쉬웠던 부분	금융 정보는 화면에 보여줄 데이터가 많아, 정보 우선순위와 가독성을 더 깊게 설계할 필요가 있었음
개선 방안	사용자 행동 로그, 네트워크 호출 수, 화면 로딩 시간 같은 정량 지표를 추가하여 개선 효과를 명확히 검증
배운 점	API 호출 구조는 앱 성능과 UX에 직결되며, Repository와 UI State 설계가 성능 최적화의 핵심임을 체감



SSABREE TIME

기간: 2026.01.06 ~ 2026.02.12 / 인원: 6명

PROJECT OVERVIEW

SSAFY 교육생을 위한 커뮤니티·모집·쪽지·알림 통합 모바일 서비스

- **목표:** 교육생 간 정보 공유와 팀원 모집이 여러 채널에 흩어져 있는 문제를 해결하기 위해, 모집과 소통을 하나의 앱으로 통합
- **배경:** 스터디 및 프로젝트 모집, 실시간 소통, 알림 수신을 일관된 모바일 사용자 흐름으로 제공할 필요성 대두

MY ROLE & CONTRIBUTION

- **주요 담당:** 그룹 찾기/상세, 팀 지원, 멤버 관리 화면 구현 및 마이페이지, 포트폴리오 기능 개발
- **실시간 소통:** 쪽지 버그 수정, 게시판 이미지 업로드 개선, 알림 관련 QA 진행
- **예외 처리:** 모집 종료, 중복 지원 등 실제 사용자 흐름에서 발생하는 예외 상황 대응 및 버그 수정

SERVICE STATUS

현재 운영 중 (유저 160+명)

160+

TOTAL USERS

LIVE

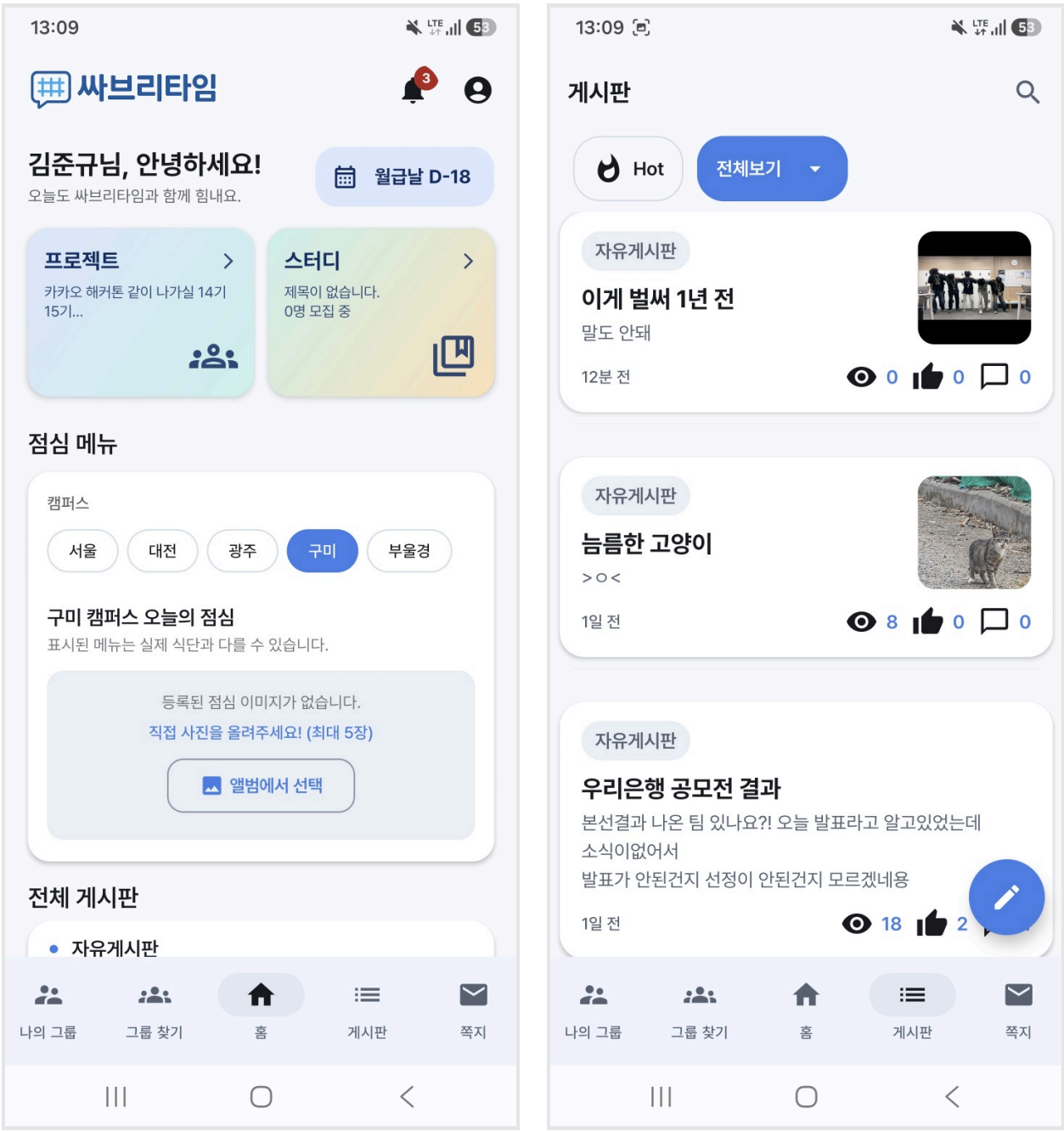
SERVICE STATUS

100%

PUSH SUCCESS

VISUAL PRESENTATION

실제 모바일 화면 배치 영역



SSABREE TIME

TECHNOLOGY

CONTEXT OF APPLICATION

Kotlin / Compose모집글, 상세, 쪽지, 마이페이지처럼 상태 변화가 잦은 화면을 선언형 UI로 유연하게 구현

MVVM지원 상태, 모집 종료 여부, 멤버 관리 상태를 ViewModel에서 일관되게 관리

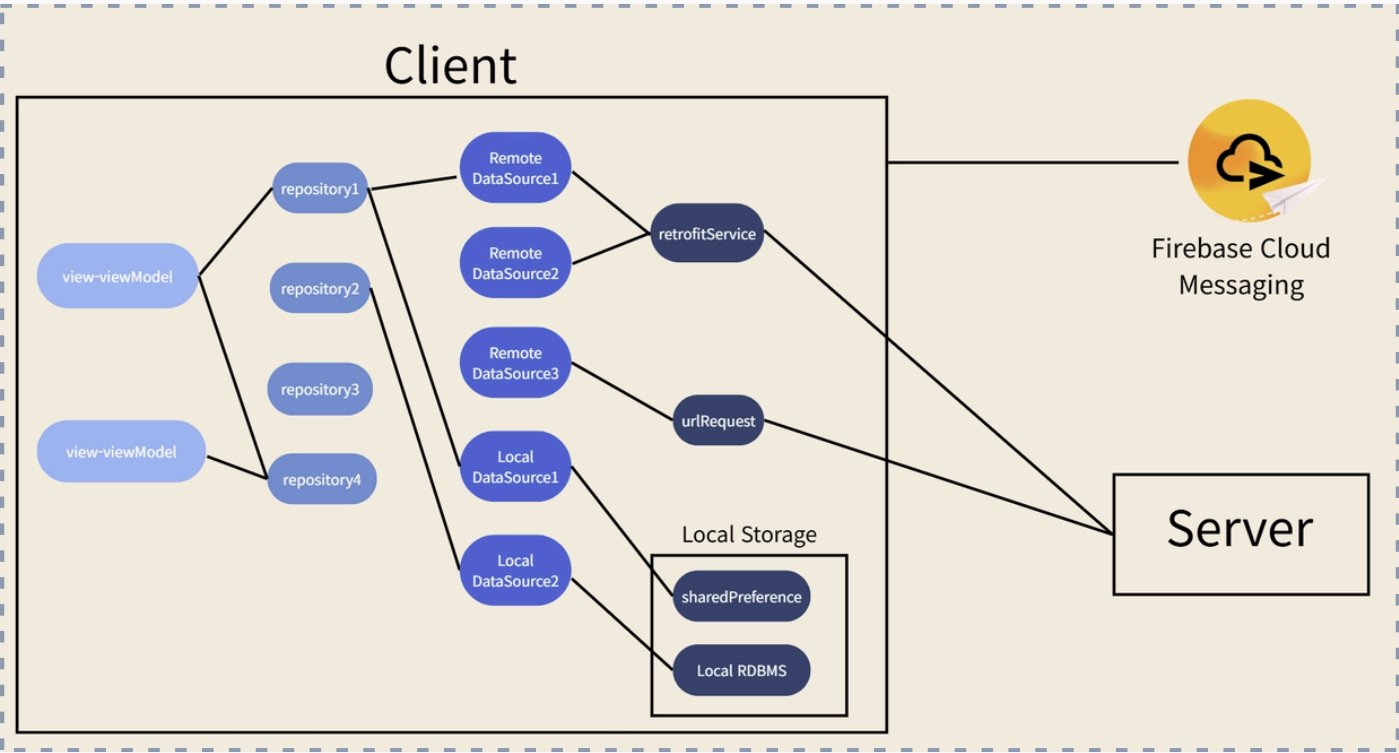
OkHttp WebSocket실시간 소통이 중요한 쪽지 및 메시지 흐름을 저지연 실시간 양방향 통신으로 처리

FCM앱이 백그라운드 상태이거나 종료된 상황에서도 중요한 알림을 놓치지 않도록 적용

Encrypted SharedPreferences사용자 인증 토큰(JWT)을 로컬에 안전하게 암호화하여 저장하고 자동 로그인 흐름 구현

KEY IMPLEMENTATIONS

- **그룹 및 지원:** 팀 목록 조회, 그룹 상세 기능 구현 및 팀 지원 스켈레톤 화면 설계
- **예외 및 흐름:** 모집 종료 상태 처리 및 중복 지원 방지 로직 개선
- **마이페이지:** 마이페이지 및 포트폴리오 기능 구현 및 로딩 속도 개선
- **미디어 및 안정성:** 게시판 사진 등록 제한 개선(최대 5개) 및 쪽지 버그 수정, QA 기반 기능 안정화



SSABREE TIME

PROBLEM STATEMENT

그룹 지원 중복 처리 및 모집 종료 상태 UI 일관성 결여

사용자가 이미 지원한 그룹에 중복 지원이 가능하거나, 모집이 종료된 상태임에도 지원 버튼이 활성화되어 있는 등 예외 흐름 제어가 미흡했습니다. 서버 상태와 화면 상태가 여러 컴포넌트에 분산되어 있어 일관된 UI 반영이 어려웠습니다.

IMPROVEMENT PROCESS

STAGE	ACTION & DETAILS
01. 문제 진단	그룹 목록, 상세, 지원 화면에서 필요한 상태 값을 구분하고 UI 영향도 분석
02. 전략 수립	지원 가능 여부와 모집 종료 여부를 화면 상태로 분리하고 조건별 버튼 문구 정의
03. 기술 적용	ViewModel에서 단일 상태(UI State)를 관리하고 Compose UI에서 조건부 렌더링 적용
04. 검증	중복 지원 차단, 모집 종료 버튼 비활성화, 로그아웃 사유 다이얼로그 등 QA 시나리오 검증

BEFORE & AFTER

Before 지원 가능 여부가 모호하여 중복 지원이 발생하고, 모집 종료 상태에서도 버튼이 활성화되어 사용자 혼란 유발

After 모집 종료, 지원 완료, 지원 가능 상태가 화면에 명확히 반영되어 중복 지원 원천 차단 및 UX 신뢰도 향상

SSABREE TIME

PROJECT ACHIEVEMENTS

- **실사용자 흐름 중심 개발:** 커뮤니티, 모집, 쪽지, 알림처럼 실사용 흐름이 길고 예외 케이스가 많은 Android 기능의 성공적 구현
- **실시간 통신 아키텍처 이해:** WebSocket과 FCM을 유기적으로 다루며 foreground/background 상황에 따른 알림 흐름의 완벽한 제어
- **QA 기반 품질 안정화:** 중복 지원, 모집 종료, 쪽지 오류, 이미지 업로드 제한 등 실제 사용자 관점의 크리티컬 버그 수정
- **통합 도메인 구현:** 마이페이지, 포트폴리오, 그룹 기능 등 여러 도메인 화면을 일관된 아키텍처로 통합 구현

RETROSPECTIVE

구분	내용
아쉬웠던 부분	실시간 통신과 알림은 앱 화면 밖에서도 동작하므로 테스트 케이스를 더 세분화할 필요가 있었음
개선 방안	WebSocket 연결 상태, 알림 수신 상태, 인증 만료 상황을 테스트 시나리오로 문서화하여 유지보수성 향상
배운 점	사용자 커뮤니케이션 기능은 단순 메시지 전송보다 상태 동기화, 예외 처리, 알림 타이밍이 핵심임을 체득